

Inferencia Eficiente: vLLM y TGI

Módulo 8 · Ingeniería de Modelos y LLMOps

PagedAttention, continuous batching, vLLM vs TGI vs TensorRT-LLM, cuantización y árbol de decisión para inferencia.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Inferencia Eficiente: vLLM y TGI

Cuando una empresa decide auto-hostear un modelo de lenguaje en lugar de usar una API de proveedor, la elección del servidor de inferencia es tan importante como la elección del modelo. El mismo modelo Llama 3 8B puede servir 120-160 consultas por segundo con vLLM, o 3 consultas por segundo con la implementación ingenua de Hugging Face Transformers generate(). La diferencia no es el modelo: es la arquitectura del servidor de inferencia.

Un estudio de noviembre 2025 (arXiv:2511.17593) realizó una comparación exhaustiva de vLLM y HuggingFace TGI en múltiples configuraciones con modelos LLaMA-2 de 7B a 70B. El resultado: vLLM alcanza hasta 24x mayor throughput que TGI bajo cargas de alta concurrencia gracias a PagedAttention, mientras que TGI v3 demuestra latencias de cola menores en escenarios de usuario único y es hasta 13x más rápido en prompts muy largos con prefix caching. La elección correcta depende del perfil de carga del sistema.

Los frameworks de inferencia LLM

vLLM: PagedAttention para máximo throughput

vLLM (UC Berkeley) introduce PagedAttention, una implementación de la atención que gestiona el KV cache como memoria virtual paginada en lugar de un buffer contiguo por secuencia. Esto elimina la fragmentación de memoria y permite empaquetar muchas más secuencias concurrentes en la misma VRAM. Combinado con continuous batching (nuevas solicitudes se incorporan dinámicamente al batch en curso en lugar de esperar el batch completo), vLLM logra una utilización de GPU del 85-92% vs el 68-74% de TGI. Throughput en Llama 2 7B: ~120-160 req/sec. API compatible con OpenAI: fácil migración desde APIs de proveedor.

TGI: producción y ecosistema Hugging Face

TGI (HuggingFace Text Generation Inference) es el framework enfocado en producción empresarial con características operativas maduras: telemetría con OpenTelemetry y métricas Prometheus integradas, prefix caching para prompts largos (13x más rápido que vLLM en prompts de >200K tokens según benchmarks de TGI v3), mejor compatibilidad con el ecosistema Hugging Face, y mayor estabilidad y previsibilidad para cargas de concurrencia media. Para stacks ya invertidos en HuggingFace, TGI es la elección natural.

TensorRT-LLM: máxima optimización NVIDIA

TensorRT-LLM compila el modelo a un motor optimizado para el hardware NVIDIA específico (similar a la compilación AOT), produciendo la menor latencia posible. Muy bajo time-to-first-token, menor latencia single-request que vLLM. El inconveniente: requiere compilación por modelo y forma, curva de aprendizaje alta, y mayor coste de mantenimiento. Para organizaciones con infraestructura NVIDIA dedicada y requisitos de latencia ultra-baja, es el framework óptimo.

Ollama y llama.cpp: desarrollo local y edge

Ollama y llama.cpp están diseñados para inferencia en CPU o GPUs de consumo. Ollama proporciona una interfaz simple para ejecutar modelos en local (MacBook, máquinas sin GPU dedicada). llama.cpp maximiza la eficiencia en CPU con cuantización agresiva. Ambos son la herramienta correcta para desarrollo local, demos, y edge computing, no para producción de alto volumen. Throughput en concurrencia alta: 1-3 req/sec vs 120-160 de vLLM.

SECCIÓN 3 · CÓMO FUNCIONA

Los mecanismos de optimización de inferencia

Continuous batching: el breakthrough de vLLM

El batching tradicional espera a que todas las secuencias de un batch terminen antes de procesar el siguiente. Esto produce subutilización de GPU cuando las secuencias tienen longitudes muy distintas (las cortas terminan y la GPU queda ociosa esperando las largas). Continuous batching incorpora nuevas secuencias al batch tan pronto como hay capacidad disponible, manteniendo la GPU ocupada continuamente. Esto se traduce directamente en mayor throughput y mejor utilización de GPU.

Quantización para inferencia: INT8 y INT4

La cuantización reduce el tamaño de los pesos del modelo para inferencia: FP16 → INT8 (2x reducción de VRAM, <1% pérdida de calidad en la mayoría de tareas), INT8 → INT4 (4x reducción, 1-5% pérdida). GGUF (formato llama.cpp) y GPTQ son los formatos estándar de cuantización. Llama 3 70B en FP16 requiere 140GB VRAM; en Q4 GGUF requiere ~35GB. Para inferencia de alta concurrencia, FP8 (soportado por H100) reduce el KV cache a la mitad manteniendo calidad comparable a BF16.

Speculative decoding: acelerar con borradores

Speculative decoding usa un modelo draft pequeño para generar un borrador de K tokens, que luego el modelo principal verifica en paralelo (en lugar de generar un token a la vez). Si el modelo principal acepta el borrador, se ahorra el tiempo de K llamadas autoregresivas. Para modelos grandes con bajo batch size (pocos usuarios concurrentes), speculative decoding puede reducir la latencia un 2-3x. vLLM soporta speculative decoding nativo.

SECCIÓN 4 · EJEMPLOS REALES

- vLLM en producción para asistente de código (startup): Llama 3.1 8B en 2x A100 80GB con vLLM. Throughput: 3.245 tokens/seg en LLaMA-2-70B con tensor parallelism en 4 GPUs (arXiv:2511.17593). Para el 8B, ~120 req/sec de throughput. Coste de infraestructura: 2x A100 en cloud ~\$6/hora vs \$600/día en tokens de API para el mismo volumen. Break-even: 24 días.
- TGI v3 para asistente conversacional con contextos largos (empresa legal): el asistente maneja conversaciones con historiales de 150K+ tokens (expedientes completos). TGI v3 con prefix caching sobre los expedientes (que son el mismo contexto para todo el hilo de conversación): 13x más rápido que vLLM en time-to-first-token para estos contextos ultra-largos.

- Multi-LoRA serving en producción (empresa SaaS con múltiples clientes): Llama 3 8B base + 12 adaptadores LoRA para 12 clientes diferentes. vLLM carga el modelo base una vez (16GB VRAM) y los adaptadores LoRA (~50MB cada uno). Cada request especifica qué adaptador usar. Coste de infraestructura: 1 GPU en lugar de 12 instancias separadas.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar `torch generate()` en producción. La implementación naive de Hugging Face `generate()` sin un servidor de inferencia dedicado no tiene continuous batching ni optimización de KV cache. Bajo cualquier concurrencia, el throughput es 10-100x menor que con vLLM o TGI. Siempre usa un servidor de inferencia para producción.

Error 2: No testear bajo carga representativa antes del despliegue. Las métricas de latencia y throughput bajo una sola solicitud no son representativas de producción con decenas de concurrentes. Realiza load testing con herramientas como locust o k6 con un perfil de carga realista antes de dimensionar la infraestructura.

Error 3: Elegir el framework solo por el throughput máximo sin considerar las características operativas. vLLM tiene mejor throughput pero TGI v3 tiene mejor telemetría integrada, mejor soporte para operaciones, y mejor comportamiento con contextos muy largos. Para equipos sin experiencia en ML infrastructure, TGI puede ser más adecuado a pesar de un throughput algo menor.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para arrancar vLLM: `pip install vllm`. Servidor con API OpenAI compatible: `python -m vllm.entrypoints.openai.api_server --model meta-llama/Llama-3.1-8B-Instruct --tensor-parallel-size 1`. El servidor expone el mismo API que OpenAI, facilitando la migración de código que usa el cliente OpenAI. Para múltiples LoRA adaptors: añadir `--enable-lora --max-lora-rank 32` y especificar el adaptador en cada request con el campo `model`.

Para benchmarking de tu modelo específico en tu hardware: usa el benchmark de vLLM (`python benchmarks/benchmark_throughput.py`) con tu perfil de carga real (prompt lengths y generation lengths representativos). Los benchmarks sintéticos no predicen bien el throughput en condiciones reales si el perfil de longitudes es muy distinto. El objetivo típico para producción: latencia P99 < 3s, throughput > 50 req/sec para un modelo 7-8B en una GPU A100.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T8 (GPUs y aceleración): los servidores de inferencia optimizan la utilización de GPU a nivel de software.
- M8-T5 (Gestión de tokens y costes): el servidor de inferencia determina el coste de tokens en despliegues autohosteados.
- M9 (Infraestructura y despliegue): vLLM y TGI son el corazón del stack de despliegue de LLMs.

SECCIÓN 8 · RESUMEN EJECUTIVO

vLLM (UC Berkeley) usa PagedAttention + continuous batching: hasta 24x mayor throughput que TGI bajo alta concurrencia, 85-92% utilización de GPU, 120-160 req/sec

en 7B. TGI v3 (HuggingFace) tiene mejor telemetría operativa, prefix caching y es 13x más rápido en prompts ultra-largos. TensorRT-LLM: menor latencia single-request en NVIDIA dedicado. Ollama/llama.cpp: desarrollo local y edge. Árbol de decisión: alta concurrencia → vLLM; contextos muy largos + HF ecosystem → TGI; NVIDIA+latencia → TensorRT-LLM; local/edge → Ollama. Arrancar vLLM: `python -m vllm.entrypoints.openai.api_server --model modelo`.